

Assignment-2.3

Batch-05

K.Amulya

2303A51242

Task-01: Generate Python code that reads a text file and counts word frequency, then explains the code.

CODE:

```
import collections
import re
import os
def count_word_frequency(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as file:
            # Read the file and convert to lowercase for uniform counting
            text = file.read().lower()
            # Use regex to find words (ignoring punctuation)
            words = re.findall(r'\b\w+\b', text)
            # Count frequencies
            word_counts = collections.Counter(words)
            # Print the top 10 most common words
            print(f"{'Word':<15} | {'Frequency':<10}")
            print("-" * 28)
            for word, count in word_counts.most_common(10):
                print(f"{'word':<15} | {'count':<10}")
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")
if __name__ == "__main__":
    # 1. Define the filename
    filename = 'test_log.txt'
    # 2. Create a dummy file so the script has something to read
    with open(filename, 'w') as f:
        f.write("System check. System check. Error found in system. Error: Timeout. Check connection.")
    # 3. Run the function
    print(f"Analyzing {filename}...\n")
    count_word_frequency(filename)
    # Optional: Clean up created file
    # os.remove(filename)
```

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\kamul> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe c:/Users/kamul/assignment2.py
Analyzing test_log.txt...

Word          | Frequency
-----
system        | 3
check         | 3
error         | 2
found         | 1
in            | 1
timeout       | 1
connection    | 1
PS C:\Users\kamul> 
```

Ctrl+K to generate command

EXPLANATION:

1. **Opens file:** Safely reads the text using a context manager.
2. **Standardizes:** Converts all text to lowercase for consistent counting.
3. **Cleans:** Uses Regex to extract only alphabetic words, removing punctuation.
4. **Tallies:** Uses Counter to instantly map words to their frequency.
5. **Sorts:** Outputs a list ordered from the most to least frequent words.

TASK-02: File Operations Using Cursor AI

```
1  Ctrl+L to chat, Ctrl+K to generate
2  # Create a new text file
3  with open('new_file.txt', 'w') as f:
4      f.write('This is a new text file.\n')
5  # Append more content
6  with open('new_file.txt', 'a') as f:
7      f.write('Adding another line to the file.\n')
8  # Read and print the content
9  with open('new_file.txt', 'r') as f:
10     content = f.read()
11     print("Contents of new_file.txt:")
12     print(content)
13
14
```

OUTPUT:

```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\kamul> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe c:/Users/kamul/assignment2.py
Contents of new_file.txt:
This is a new text file.
Adding another line to the file.

PS C:\Users\kamul> []
```

EXPLANATION:

1. The first with `open('new_file.txt', 'w')` creates a new text file and writes a line to it.
2. The 'w' mode overwrites the file if it already exists.
3. The second with `open('new_file.txt', 'a')` opens the same file in append mode to add more content without deleting existing data.
4. The third with `open('new_file.txt', 'r')` opens the file in read mode and reads all its contents.
5. Finally, the content of the file is printed to the console.

TASK-03: CSV Data Analysis

```
[5] import pandas as pd
import numpy as np

# Create a simple DataFrame
data = {
    'Category': ['A', 'B', 'A', 'C', 'B'],
    'Value1': [10, 20, 15, 25, 12],
    'Value2': [100, 150, 120, 180, 110],
    'Value3': [1.1, 2.2, 1.5, 3.0, 1.8]
}
dummy_df = pd.DataFrame(data)

# Save the DataFrame to a CSV file
csv_filename = 'my_data.csv'
dummy_df.to_csv(csv_filename, index=False)
print(f'{csv_filename} created successfully!')
display(dummy_df.head())
```

... 'my_data.csv' created successfully!

	Category	Value1	Value2	Value3
0	A	10	100	1.1
1	B	20	150	2.2
2	A	15	120	1.5
3	C	25	180	3.0
4	B	12	110	1.8

Now, let's read the created CSV file and calculate the mean, minimum, and maximum for its numerical columns.

```
[6] import pandas as pd
✓ 0s

try:
    # Read the CSV file we just created
    df_from_csv = pd.read_csv('my_data.csv')
    print("CSV file 'my_data.csv' loaded successfully!")
    display(df_from_csv.head())

    # Select only numerical columns for calculations
    numeric_df_from_csv = df_from_csv.select_dtypes(include=['number'])

    if not numeric_df_from_csv.empty:
        # Calculate mean
        mean_values_csv = numeric_df_from_csv.mean()
        print("\nMean values for numerical columns from CSV:")
        display(mean_values_csv)

        # Calculate minimum
        min_values_csv = numeric_df_from_csv.min()
        print("\nMinimum values for numerical columns from CSV:")
        display(min_values_csv)

        # Calculate maximum
        max_values_csv = numeric_df_from_csv.max()
        print("\nMaximum values for numerical columns from CSV:")
        display(max_values_csv)
```

```
[6] else:
✓ 0s     print("No numerical columns found in the CSV DataFrame to calculate mean, min, and max.")

except FileNotFoundError:
    print("Error: 'my_data.csv' not found. Please make sure the file is in the correct directory.")
except Exception as e:
    print(f"An error occurred while reading the CSV: {e}")
```

... CSV file 'my_data.csv' loaded successfully!

	Category	Value1	Value2	Value3
0	A	10	100	1.1
1	B	20	150	2.2
2	A	15	120	1.5
3	C	25	180	3.0
4	B	12	110	1.8

Mean values for numerical columns from CSV:

0

Value1 16.40

Value2 132.00

Value3 1.92

dtype: float64

```
dtype: float64
...
Minimum values for numerical columns from CSV:
0
Value1    10.0
Value2    100.0
Value3     1.1

dtype: float64

Maximum values for numerical columns from CSV:
0
Value1    25.0
Value2    180.0
Value3     3.0

dtype: float64
```

TASK-04: Sorting Lists – Manual vs Built-in

```
C: > Users > kamul > assignment2.py > bubble_sort

1  def bubble_sort(arr):
2      n = len(arr)
3      # Traverse through all array elements
4      for i in range(n):
5          # Last i elements are already in place
6          for j in range(0, n - i - 1):
7              # Traverse the array from 0 to n-i-1
8              # Swap if the element found is greater than the next element
9              if arr[j] > arr[j + 1]:
10                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
11         return arr
12 data = [64, 34, 25, 12, 22, 11, 90]
13 sorted_data = bubble_sort(data)
14 print("Sorted Array:", sorted_data)
15
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\kamul> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe c:/Users/kamul/assignment2.py
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\kamul>
```

```
> Users > kamul > assignment2.py > ...
```

```
1 numbers = [5, 2, 9, 1, 5, 6]
2 # Sorts the list in-place
3 numbers.sort()
4 print(numbers)
5
```

```
PS C:\Users\kamul> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe c:/Users/kamul/assignment2.py
[1, 2, 5, 5, 6, 9]
PS C:\Users\kamul>
```

EXPLANATION:

Speed (Math): Bubble Sort is $O(n^2)$, meaning if you double the data, it takes 4x longer.³ Built-in sort is $O(n \log n)$, which scales much better. For 10,000 items, Bubble Sort does ~100 million operations; Built-in sort does ~133,000.

Speed (Code): Bubble Sort runs in interpreted **Python** (slow). Built-in sort runs in pre-compiled **C** (fast).

Intelligence: Bubble Sort blindly swaps neighbors.⁴ Built-in sort analyzes the data to find existing patterns and merges them efficiently

